INTRODUCING 0.2.1

intermine314

0.2.1

Familiar InterMine.
Modern performance.

Keep your workflows.
Unlock columnar analytics.

- PARQUET
- DUCKDB
- ARROW
- POLARS

PYTHON 3.14.5+

A familiar way to work.

The historical Service interface stays at the center.

YOUR EXISTING IMPORT

```
from intermine.webservice import Service
```

YOUR NEW IMPORT

```
from intermine314.webservice import Service
```

Queries Templates Lists Models Registry

Update dependency declarations; review documented API and dtype differences.

Closer to the InterMine code you already use.

0.2.0 broadened support. 0.2.1 restores more original behavior.

Earlier releases

Narrower legacy support

More migration work

0.2.0

Broader API coverage

Queries, XML and templates

Lists, summaries and registry

Legacy row / object defaults

0.2.1

Closer legacy behavior

Optional pandas output

Result sizing and mutability

Live result customization

More familiar calls. Fewer changes to established workflows.

Compatibility goes beyond method names.

Legacy behavior at the boundary. Native analytics underneath.

Existing code relies on…	0.2.0	0.2.1
dataframe() output	Polars	pandas in legacy mode*
Result length / truth	Unsized streams	Original sizing behavior
Mutable row metadata	Shared / immutable	Mutable in legacy mode
Live customization	Bound at stream opening	Changes respected

Also restored: public encoders and typed constraint values.

*Requires the pandas extra; output uses Arrow dtypes. Native mode retains Polars.

FASTER FROM DATA TO ANSWERS

Spend less time moving and reading data.

One million annotation rows. Same data. Same analytical result.

32.2×

Faster aggregation

CSV + pandas

Parquet + DuckDB

2.6×

Faster export

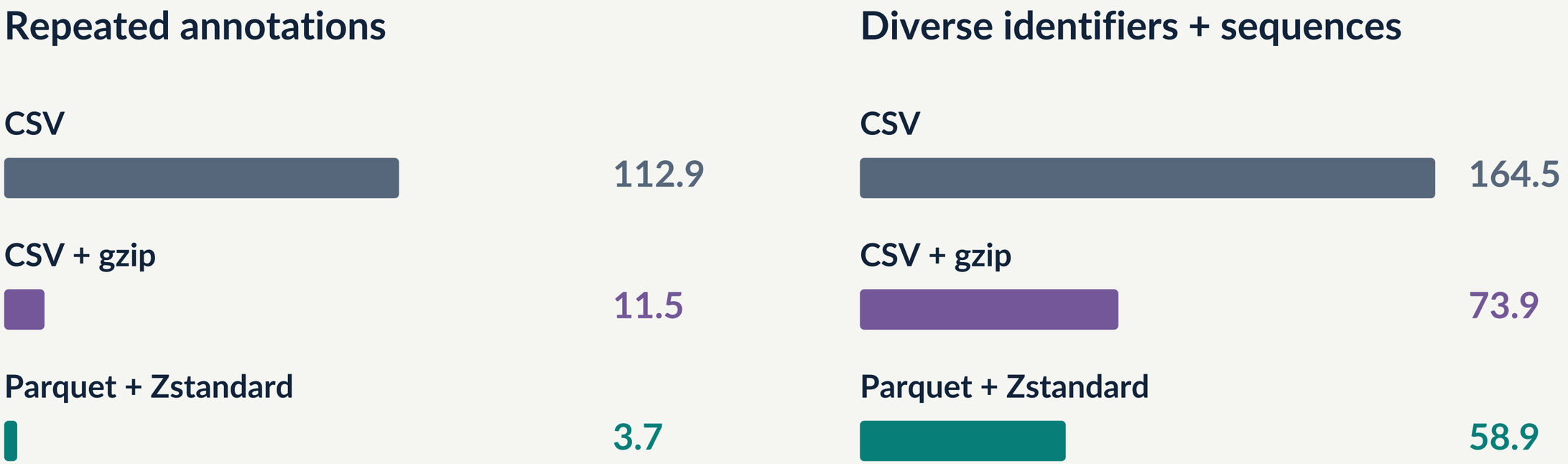
Original dataframe → CSV

intermine314 → Parquet

Synthetic local benchmark; warm cache. Small jobs and streaming CSV can favor the original client.

64–97% less disk than plain CSV.

Measured at one million rows across two data profiles.



Typed columns. Selective reads. A reusable analytical dataset.

Use your CPU. Manage your memory.

Stage-aware workers and bounded page queues.

16

Default native CPU budget

Other profiles: 8 · 6 · 4 · 2

4
HTTP

4
Arrow CPU

16
DuckDB SQL

12
Polars

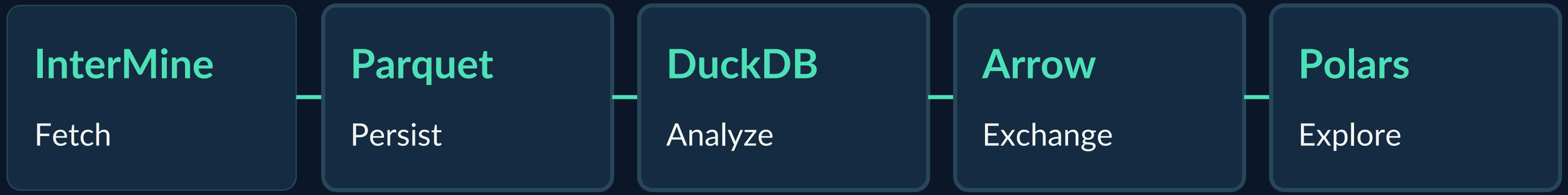
38% lower export peak RSS

1M annotation rows, versus original pandas → CSV.

Diverse data used 7% more RSS; streaming CSV stayed smallest. Stage allocations are not additive.

Query once. Keep getting value.

Python 3.14.5+ with native columnar engines.



Validated pages. Typed data. Failure-safe export.

One shared pipeline for legacy and native interfaces.

Built on tested public contracts.

Validated against original InterMine and fresh package installations.

2,805

Source tests passed

1,003

Installed regressions

2 mines

Live read-only checks

338 passing API scopes

Up from 331 in 0.2.0.

460 inventoried symbols; 122 have tested departures. Counts describe scoped checks, not complete equivalence.

Keep your InterMine expertise.

Expand what your data can do.

Familiar APIs.

Reusable Parquet.

Fast native analytics.

Explore the project

Compatibility guide

Benchmark evidence

0.2.1 release preview · Full methods and qualifications accompany this deck.